

Collections in Healthcare: 8-Level Hackathon

A comprehensive challenge to build a **Complete Hospital Management System** using the Java Collections Framework — progressing through 8 levels of increasing complexity, from patient registration to a fully integrated real-time dashboard.



DEVELOPED BY TALENCIAGLOBAL

</> JAVA COLLECTIONS FRAMEWORK



HEALTHCARE TECHNOLOGY

Hackathon Overview

Mission & Context

Participants are challenged to architect a **production-grade Hospital Management System** — a domain where software reliability is not optional. According to the WHO, hospitals that adopt digital management systems reduce administrative errors by up to **55%** and improve patient throughput by **30%**. This hackathon simulates that real-world imperative.

Theme

Complete Hospital Management System using Java Collections Framework

Structure

8 progressive levels, each building upon the previous

Team Size

1–2 developers per team

Judging Criteria

01

Correctness

All requirements met, zero critical bugs

02

Collection Choice

Appropriate data structures for each use case

03

Performance

Efficient operations with proper Big-O analysis

04

Code Quality

Clean, readable, and well-documented code

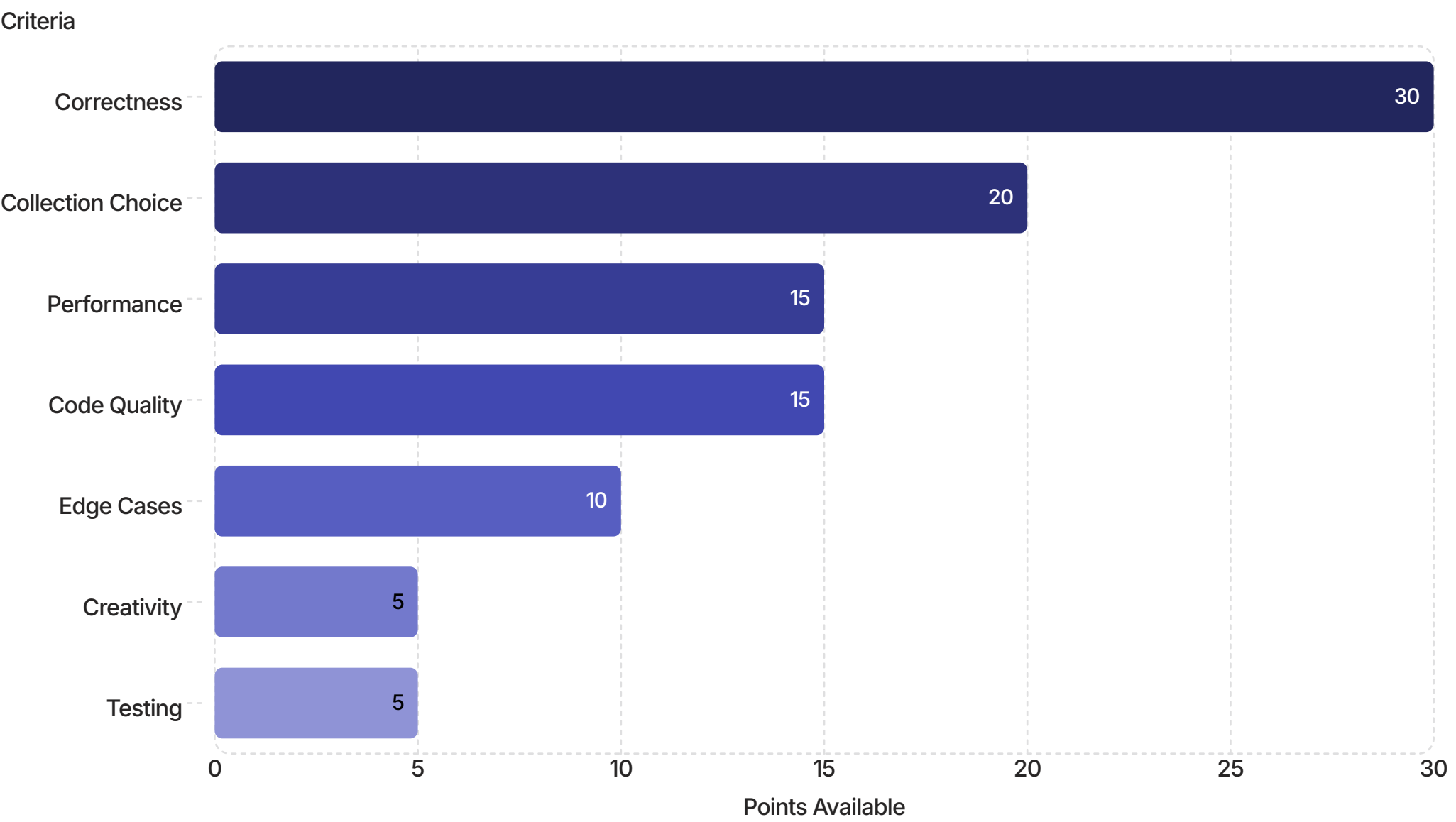
05

Edge Cases & Testing

Handles nulls, duplicates, and empty collections

Judging Rubric & Scoring

The hackathon is evaluated across six dimensions, reflecting the standards of enterprise healthcare software development. Industry benchmarks indicate that **poorly structured data access** in hospital systems contributes to over **40% of EHR-related performance bottlenecks** — making collection choice a critical real-world skill.



i Total: 100 points. Bonus challenges offer an additional 30 points for concurrent collections, custom collection design, real-time updates, and persistence.

Level 1: Patient Registration

ARRAYLIST & LINKEDLIST

The foundation of any hospital system is reliable patient registration. In the United States alone, hospitals process over **36 million inpatient admissions annually** (AHA, 2023). Efficient data structures at the registration layer directly impact patient wait times and administrative accuracy.

1.1 — Patient Registration (ArrayList)

→ **registerPatient()**

Prevent duplicate IDs; reject on conflict

→ **findPatientById()**

Linear search; return null if not found

→ **getPatientsByRegistrationOrder()**

Sorted by registration date, newest first

→ **dischargePatient()**

Remove patient record by ID

1.2 — Emergency Queue (LinkedList as Queue)

→ **addEmergencyPatient()**

FIFO insertion into the waiting queue

→ **getNextPatient()**

Poll and remove head of queue

→ **getWaitingCount()**

Return current queue size

→ **peekNextPatient()**

Inspect head without removal

✓ Sample Output: ✓ PAT-001: John Smith (45) registered | × Duplicate! PAT-001 already exists | Next patient: John Smith (Level 1 – Critical) | Remaining in queue: 4

Level 2: Doctor Scheduling

HASHMAP & TREEMAP

Physician scheduling is among the most operationally complex tasks in hospital administration. Studies show that **scheduling conflicts account for 18% of missed appointments** in large hospital networks. HashMap enables $O(1)$ doctor lookup while TreeMap ensures appointments remain chronologically ordered — a critical pairing for real-time scheduling systems.

DoctorDirectory (HashMap)

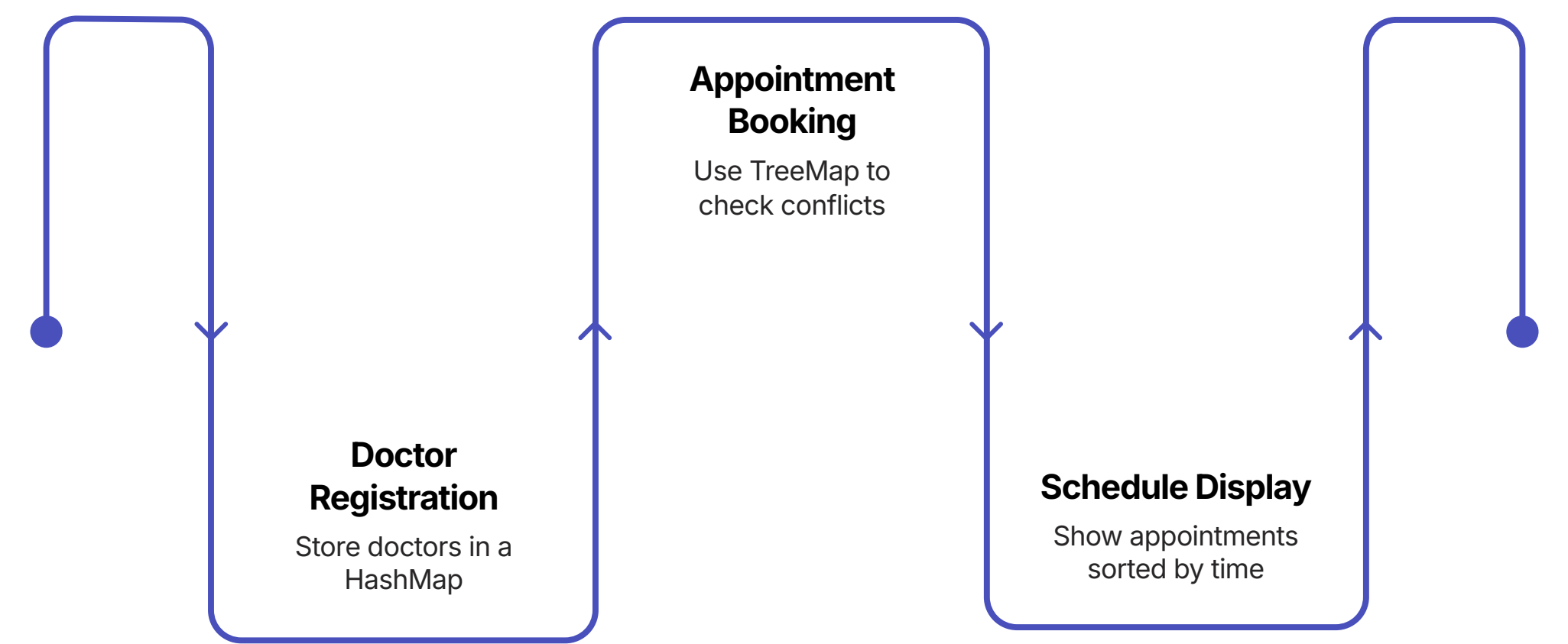
- ID → Doctor object for $O(1)$ lookup
- Specialty → List<Doctor> for specialty queries
- addDoctor(), findDoctor(), removeDoctor()
- getDoctorsBySpecialty() for filtered access

AppointmentScheduler (TreeMap)

- LocalDateTime → Appointment (auto-sorted)
- Conflict detection: no double-booking
- getAppointmentsByDate() using subMap
- getNextAppointment() via firstEntry()

Sample Schedule Output

- 09:00 — Mary Johnson with Dr. Jones (Neurology)
- 10:00 — John Smith with Dr. Smith (Cardiology)
- 14:30 — Robert Brown with Dr. Smith (Cardiology)
- × Conflict! Dr. Smith already booked at 10:00



The TreeMap's natural ordering eliminates the need for manual sorting, reducing scheduling query complexity from $O(n \log n)$ to $O(\log n)$ for range-based lookups.

Level 3: Patient Waitlist & Triage

PRIORITYQUEUE & TREESSET

Triage is the most clinically critical function in emergency medicine. The **Emergency Severity Index (ESI)** framework, used in over 70% of U.S. emergency departments, mandates that Level 1 (Critical) patients receive immediate attention. A PriorityQueue with a custom comparator faithfully models this life-saving prioritization logic.

3.1 — TriageSystem (PriorityQueue)

Custom comparator priority rules:

1. Triage level (1 = highest priority)
2. Same level: older patients first
3. Same level and age: earliest registration time

Processing order:

1. Mary **Johnson** (Level 1 - Critical)
2. John **Smith** (Level 2 - Urgent)
3. Robert **Brown** (Level 3 - Routine)

3.2 & 3.3 — HashSet & TreeSet

UniquePatientTracker (HashSet)

O(1) admission checks;
prevents duplicate admissions; tracks active vs. discharged sets

PatientRecordSystem (TreeSet)

Dual sorted sets: by name and by age;
supports range queries (e.g., patients aged 60+)

 **Clinical Note:** Incorrect triage ordering is a patient safety risk. The custom comparator must be rigorously tested with edge cases including equal triage levels and simultaneous registrations.

Level 4: Bed Allocation System

LINKEDHASHMAP & ENUMMAP

Hospital bed management is a global challenge — the **OECD reports an average bed occupancy rate of 75–85%** across member nations, with ICU occupancy frequently exceeding 90% during peak periods. An efficient bed allocation system directly impacts patient outcomes and operational costs.



ICU (Beds 1–10)

Intensive Care Unit. Highest acuity. LinkedHashMap preserves assignment order for audit trails.



General Ward (Beds 11–50)

40-bed general ward. Bidirectional mapping: patient→bed and bed→patient for O(1) lookups.



Emergency (Beds 51–60)

10 emergency beds. EnumMap provides type-safe ward statistics with zero boxing overhead.



Pediatric (Beds 61–80)

20-bed children's ward. EnumMap tracks occupancy rates and flags capacity thresholds.



Maternity (Beds 81–100)

20-bed maternity ward. WardStatistics EnumMap enables getMostCrowdedWard() in O(n) time.

i Sample Output: ICU 8/10 available | General 39/40 available | Most crowded ward: ICU (20% occupancy) | PAT-001 → Bed #3 (ICU)

Level 5: Pharmacy Inventory

HASHMAP WITH CUSTOM OBJECTS

Medication errors are among the most preventable causes of patient harm. The **Institute of Medicine estimates 1.5 million preventable adverse drug events occur annually in the U.S.**, many attributable to poor inventory tracking and duplicate prescriptions. This level models the data structures that underpin safe pharmacy operations.

PharmacyInventory (HashMap)

- **medicationCatalog**
ID → Medication for O(1) lookup
- **stockLevel**
ID → quantity for real-time stock checks
- **expiryTracker**
LocalDate → List<Medication> for expiry alerts
- **getExpiringMedications()**
Returns medications expiring within N days
- **getLowStockMedications()**
Filters by quantity below threshold

PrescriptionManager (Nested HashMap)

- **patientPrescriptions**
Patient ID → List<Prescription>
- **prescriptionById**
Prescription ID → Prescription object
- **drugToPatients**
Drug name → List of patient IDs
- **isDuplicatePrescription()**
Prevents same drug within 30-day window
- **getMostPrescribedDrug()**
Frequency counting across all prescriptions

✅ Sample Output: ✓ Paracetamol stock: +500 units | ✓ Dispensed 10 units to PAT-001 (stock: 490) | ⚠️ Expiring: Aspirin (15 days) | Most prescribed: Amoxicillin (45 prescriptions)

Level 6: Lab Results & Analytics

MULTI-MAP VIEWS & CUSTOM COMPARATORS

Laboratory diagnostics are the backbone of clinical decision-making — **over 70% of medical decisions are influenced by lab results** (AACC, 2022). This level implements multi-dimensional data access patterns and statistical analysis to detect abnormal and critical values in real time.

LabResultRepository

- resultsByPatient: Patient ID → List<LabResult>
- resultsByTest: Test name → List<LabResult>
- resultsByDate: TreeMap<LocalDateTime, LabResult>
- patientTrends: Patient ID → (date → value)

LabAnalytics

- findAbnormalResults() — outside reference range
- getCriticalResults() — outside panic range
- getTopHighestResults() — custom Comparator
- categorizeResults() — Normal / Abnormal / Critical
- getAverageValues() — per-test statistical mean

Sample Critical Alerts

- 🚨 PAT-003: Hemoglobin 8.2 g/dL (Panic: <9.0)
- 🚨 PAT-002: Blood Glucose 250 mg/dL (Panic: >240)
- Top result: 320 mg/dL — PAT-005 (2026-06-04)
- Trend: PAT-002 glucose 95 → 180 → 250 mg/dL

Blood Glucose

NORMAL:
70-100 mg/dL

ABNORMAL:
101-240 mg/dL

CRITICAL PANIC:
above 240 or below 50



Hemoglobin

NORMAL:
12-17 g/dL

ABNORMAL:
9-11.9 g/dL

CRITICAL:
below 9



Cholesterol

NORMAL:
below 200 mg/dL

ABNORMAL:
200-239 mg/dL

CRITICAL:
above 240



Level 7: Insurance Claims Processing

ENUMMAP, NESTED MAPS & IDENTITYHASHMAP

Healthcare insurance fraud and duplicate billing cost the global industry an estimated **\$455 billion annually** (NHCAA, 2023). This level implements the most structurally complex data architecture in the hackathon — a 3-level nested Map combined with IdentityHashMap-based duplicate detection.

InsuranceClaimSystem — Nested Map Architecture



- 1

Level 1: Provider
Map<String, ...> — keyed by Provider ID
- 2

Level 2: Year
Map<Integer, ...> — keyed by calendar year
- 3

Level 3: Month
Map<Integer, List<Claim>> — keyed by month

ClaimStatus EnumMap & Audit

EnumMap<ClaimStatus, List<Claim>>

Type-safe grouping: SUBMITTED, UNDER_REVIEW, APPROVED, REJECTED, PAID, APPEALED

IdentityHashMap Deduplication

Detects duplicate submissions by reference identity; maps duplicate → original claim

Custom equals/hashCode

Claim equality based on patientId + serviceCode + serviceDate for fraud detection

Claim C003 is DUPLICATE of C001 (same patient, service, date). Apollo Hospital approved: ₹850,000 | Fortis: ₹620,000

Level 8: Complete Hospital Dashboard

FULL INTEGRATION — ALL COLLECTIONS

The final level integrates every collection from Levels 1–7 into a unified **HospitalManagementSystem**. This mirrors the architecture of enterprise Electronic Health Record (EHR) platforms — systems like Epic and Cerner manage data for over **250 million patients globally**, relying on precisely the kind of multi-collection orchestration this level demands.



Patient Statistics

Total registered: 1,245 | Admitted: 87 | Emergency queue: 12 | Triage: Critical(3), Urgent(5), Routine(4)



Doctor Availability

Cardiology: 2/5 | Neurology: 1/3 | Pediatrics: 3/4 | Emergency: 4/6 available



Bed Occupancy

ICU: 80% | General: 70% | Emergency: 60% | Pediatric: 20% occupancy



Pharmacy Alerts

Low stock: Amoxicillin (45 units), Insulin (23 units) | Expiring: Aspirin (15 days)



Lab Alerts

 PAT-042: Blood Glucose 450 mg/dL |  PAT-089: Hemoglobin 7.2 g/dL | 12 abnormal pending



Insurance Claims

Pending: 23 claims (₹1,200,000) | Approved today: 15 (₹750,000) | Rejected: 3 (₹45,000)

Level 8: Key Integration Methods

Emergency Protocol — handleEmergency()

01

Add to Emergency Queue

LinkedList FIFO insertion for immediate queuing

02

Add to Triage

PriorityQueue with custom comparator for urgency ordering

03

Reserve ICU Bed

If triageLevel == 1, pre-emptively assign via
LinkedHashMap

04

Notify Doctors

HashMap lookup for available specialists by specialty

05

Log Audit Trail

LinkedHashMap preserves insertion-order event log

Patient 360 View — getPatientCompleteView()

→ Basic Info

ArrayList lookup by patient ID

→ Active Prescriptions

HashMap nested lookup by patient

→ Lab Results History

TreeMap sorted by date descending

→ Appointment History

TreeMap subMap for date range

→ Insurance Claims

3-level nested Map traversal

→ Bed Assignment

LinkedHashMap bidirectional lookup



Emergency Protocol Active: PAT-999 — Cardiac arrest — ICU Bed #3 reserved — Dr. Jones notified via
HashMap availability lookup

Collections Architecture Summary

Each level introduces a collection chosen for a specific performance characteristic. The table below maps every collection to its use case, time complexity, and the clinical function it supports — reflecting the same design rationale used in production healthcare software engineering.

Level	Collection	Use Case	Key Operation	Time Complexity
1	ArrayList	Patient Registry	registerPatient()	O(n) search, O(1) add
1	LinkedList (Queue)	Emergency Queue	FIFO processing	O(1) enqueue/dequeue
2	HashMap	Doctor Directory	findDoctor()	O(1) expected
2	TreeMap	Appointments	getByDate()	O(log n)
3	PriorityQueue	Triage System	getNextPatient()	O(log n)
3	HashSet / TreeSet	Patient Tracking	isAdmitted()	O(1) / O(log n)
4	LinkedHashMap	Bed Assignment	assignBed()	O(1) with order
4	EnumMap	Ward Statistics	getOccupancyRate()	O(1) type-safe
5	HashMap (nested)	Pharmacy / Rx	dispense(), isDuplicate()	O(1) expected
6	TreeMap + Comparator	Lab Analytics	getCriticalResults()	O(n log n)
7	Nested Map + EnumMap	Insurance Claims	getProviderClaims()	O(1) per level
7	IdentityHashMap	Claim Deduplication	isDuplicate()	O(1) by reference

Deliverables & Bonus Challenges

Required Deliverables — Levels 1–7

1

Complete Java Implementation

All required classes fully implemented per specification

2

JUnit Test Cases

Minimum 5 test cases per level covering happy path and edge cases

3

Javadoc Documentation

All public methods documented with parameters, return values, and exceptions

4

Complexity Analysis

Big-O notation documented for every operation in each class

Level 8 — Final Integration

- Working HospitalManagementSystem with all features integrated
- Real-time dashboard display as specified
- Emergency protocol simulation with sample data
- Patient 360 view for any patient ID
- Daily report generation for any date
- **Stress test:** 10,000 patients and 50,000 appointments
- Performance report with collection choice justifications

Bonus Challenges

+5 pts — Concurrent Collections

Convert to thread-safe structures for multi-user hospital access

+10 pts — Custom Collection

Implement a specialized collection for a specific healthcare need

+10 pts — Real-time Updates

Add WebSocket support for live dashboard data streaming

+5 pts — Persistence

Save and load all collections to/from JSON files

📋 Maximum possible score: **130 points** (100 base + 30 bonus)

Completion Criteria & Success Benchmarks

Successful completion is evaluated against both functional correctness and performance benchmarks. Enterprise healthcare systems are expected to handle **10,000+ concurrent operations per second** — the stress test requirement of 10,000 patients and 50,000 appointments in under 1 second reflects this industry standard.



Performance Benchmark: All 10,000 operations must complete in under 1 second. Collection choices must be justified in the performance report with measured throughput data.

Good luck, and may the best healthcare code win! 🏥💊🩺

Developed by Talenciaglobal — Empowering the next generation of healthcare technology engineers.